

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION WITH OPENCV

COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: *Understand video processing - motion computation - 3D vision - geometry.* (BTL 6)

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Face Detection and

Recognition Pipeline A system must process a live video stream to:

- A. Capture frames from a camera
 - B. Detect faces in each frame
 - C. Recognize faces using a pre-trained dataset
 - D. Display bounding boxes and labels in real time
- Constraints:
- Must operate continuously
 - Should minimize processing delay
 - Should handle multiple faces

Write detailed pseudocode for the complete pipeline using OpenCV concepts, including image acquisition, preprocessing, detection, recognition, and display. Ensure modular design and efficient looping structure.

ANS:

1. Real-Time Face Detection and Recognition Pipeline
Objective

Design a real-time computer vision pipeline that continuously captures video frames, detects multiple faces, recognizes them using a pre-trained dataset, and displays bounding boxes with identity labels.

Pseudocode

ALGORITHM RealTimeFaceRecognition

INPUT:

- Live camera stream
- Pre-trained face recognition dataset

OUTPUT:

- Live video with face bounding boxes and identity labels

BEGIN

MODULE 1: INITIALIZATION

- Import OpenCV library

- Open camera using VideoCapture

- Load pre-trained face detection model

 - face_detector ← Load Haar Cascade Classifier

- Load pre-trained face recognition model

 - face_recognizer ← Load trained recognition model

- Load person labels/names from the dataset

- Set detection parameters

 - scale_factor ← 1.1

 - min_neighbors ← 5

- Set recognition threshold

- IF camera cannot be opened THEN

 - Display "Camera Error"

 - STOP

- END IF

MODULE 2: CONTINUOUS FRAME ACQUISITION

WHILE camera is running DO

Capture one frame from camera

IF frame is not successfully captured THEN
CONTINUE
END IF

MODULE 3: PREPROCESSING

Resize frame to smaller resolution
frame $\leftarrow$ Resize(frame)

Convert frame from BGR to grayscale
gray $\leftarrow$ ConvertToGrayscale(frame)

Apply optional histogram equalization
gray $\leftarrow$ EqualizeHistogram(gray)

MODULE 4: FACE DETECTION

Detect all faces in the grayscale frame

faces $\leftarrow$ DetectFaces(
 gray,
 scale_factor,
 min_neighbors
)

FOR each face in faces DO

Get bounding box coordinates
x, y, width, height $\leftarrow$ face

Extract face region
face_ROI $\leftarrow$ gray[y : y+height,
 x : x+width]

MODULE 5: FACE RECOGNITION

Resize face_ROI if required

predicted_label,
confidence ←
 RecognizeFace(face_ROI)

IF confidence satisfies recognition threshold THEN
 name ← GetName(predicted_label)
ELSE
 name ← "Unknown"
END IF

MODULE 6: DISPLAY RESULT

Draw bounding rectangle around face

Display person's name above
or near the bounding box

Display confidence if required

END FOR

MODULE 7: DISPLAY FRAME

Show processed frame on screen

Wait for a very short time

IF user presses 'q' THEN
 BREAK
END IF

END WHILE

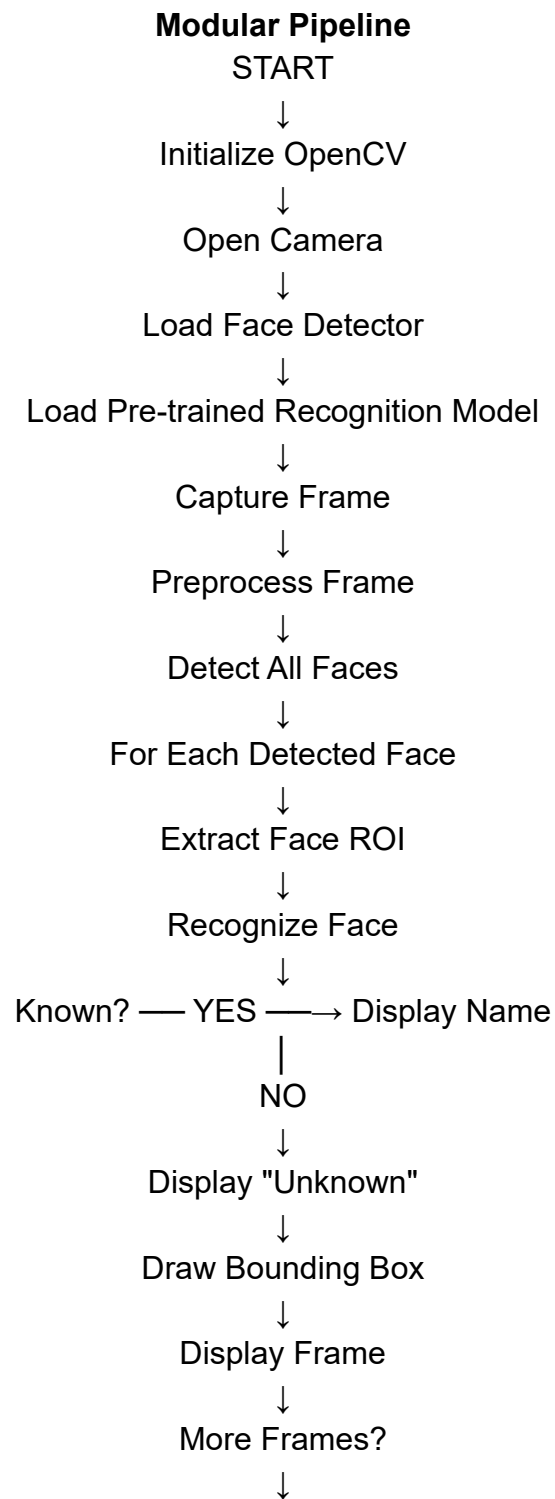
MODULE 8: TERMINATION

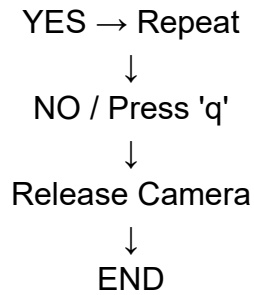
Release camera

Close all OpenCV windows

Display "Face Recognition Stopped"

END





Handling Multiple Faces

The algorithm uses a loop to process every detected face in the current frame.
 FOR each face in detected_faces DO

Extract face ROI

Recognize face

Draw bounding box

Display corresponding name

END FOR

Therefore, if three faces are present, all three faces are detected, recognized independently, and displayed with separate labels.

Efficiency Considerations

1. **Continuous loop:** The WHILE loop continuously processes live camera frames.
2. **Frame resizing:** Smaller frames reduce processing time.
3. **Grayscale conversion:** Reduces computational requirements.
4. **ROI processing:** Recognition is performed only on detected face regions.
5. **Pre-trained models:** Avoids training during real-time operation.
6. **Short display delay:** Allows continuous frame processing.
7. **Multiple-face loop:** Processes all detected faces in the same frame.
8. **Immediate termination:** Pressing q safely stops the system and releases resources.

OpenCV Functions Used

Pipeline Stage	OpenCV Concept / Function
Image acquisition	cv2.VideoCapture()
Resizing	cv2.resize()
Grayscale conversion	cv2.cvtColor()
Contrast enhancement	cv2.equalizeHist()
Face detection	Haar Cascade detectMultiScale()
Face recognition	LBPH / pre-trained recognizer

Pipeline Stage	OpenCV Concept / Function
Bounding box	cv2.rectangle()
Label display	cv2.putText()
Video display	cv2.imshow()
Key detection	cv2.waitKey()
Resource release	cap.release()
Window closing	cv2.destroyAllWindows()

Conclusion

The proposed pseudocode divides the face recognition system into modular stages: initialization, image acquisition, preprocessing, face detection, face recognition, and display. The continuous WHILE loop ensures real-time operation, while resizing and grayscale conversion reduce processing delay. The FOR loop allows multiple faces to be processed in every frame. Finally, bounding boxes and identity labels are displayed continuously until the user presses q.

2. OCR Pipeline for Document Processing

A document processing system must:

- A. Read input images containing printed text
 - B. Enhance image quality (noise removal, contrast improvement)
 - C. Segment text regions
 - D. Extract and display recognized text
- Constraints:
- Input images may be noisy and low contrast
 - Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

ANS:

OCR Pipeline for Document Processing

Objective

Design an end-to-end OCR pipeline that reads printed text from input images, improves image quality, segments text regions, extracts recognized text, and processes multiple documents efficiently.

Pseudocode

ALGORITHM OCRDocumentProcessing

INPUT:

Folder containing document images

OUTPUT:

Recognized text from each document

BEGIN

MODULE 1: INITIALIZATION

Import OpenCV library

Import OCR library

Get list of all images from input folder

IF input folder is empty THEN

Display "No images found"

STOP

END IF

MODULE 2: BATCH PROCESSING

FOR each image_file in image_folder DO

Display "Processing:", image_file

Read input image

image ← cv2.imread(image_file)

IF image is not successfully loaded THEN

Display "Image loading error"

CONTINUE

END IF

MODULE 3: IMAGE PREPROCESSING

Resize image to improve character visibility

image ← cv2.resize(image)

Convert image to grayscale


```
gray ← cv2.cvtColor(  
    image,  
    BGR_TO_GRAY  
)
```

Remove image noise

```
gray ← cv2.GaussianBlur(  
    gray,  
    kernel_size,  
    sigma  
)
```

Improve contrast

```
enhanced ← ApplyHistogramEqualization(gray)
```

IF illumination is uneven THEN

```
    enhanced ← ApplyCLAHE(enhanced)
```

END IF

MODULE 4: TEXT SEGMENTATION

Convert enhanced image into binary image

```
binary ← cv2.threshold(  
    enhanced,  
    OTSU  
)
```

IF document has uneven lighting THEN

```
    binary ← cv2.adaptiveThreshold(  
        enhanced  
    )
```

END IF

Apply morphological operations

```
binary ← MorphologicalOpening(binary)  
binary ← MorphologicalClosing(binary)
```

Detect contours

```
contours ← cv2.findContours(binary)
```

Identify possible text regions

FOR each contour in contours DO

 Calculate contour area

 IF contour area is greater than
 minimum_area THEN

 Create bounding box
 x, y, width, height $\leftarrow$
 cv2.boundingRect(contour)

 Store text region

 END IF

END FOR

MODULE 5: TEXT REGION EXTRACTION

FOR each detected text region DO

 Extract Region of Interest (ROI)

 Crop the text region from binary image

 Resize ROI if required

 Store ROI for OCR processing

END FOR

MODULE 6: OCR TEXT EXTRACTION

 recognized_text $\leftarrow$ EMPTY

FOR each text ROI DO

 Send ROI to OCR engine

text $\leftarrow$ OCR(ROI)

Add text to recognized_text

END FOR

MODULE 7: POST-PROCESSING

Remove unnecessary spaces

Remove unwanted characters

Correct obvious OCR formatting errors

Arrange recognized text in proper
reading order

MODULE 8: OUTPUT

Display recognized_text

Save recognized_text to output file

Display "Processing completed"

END FOR

MODULE 9: TERMINATION

Display "All documents processed"

END

Complete OCR Pipeline

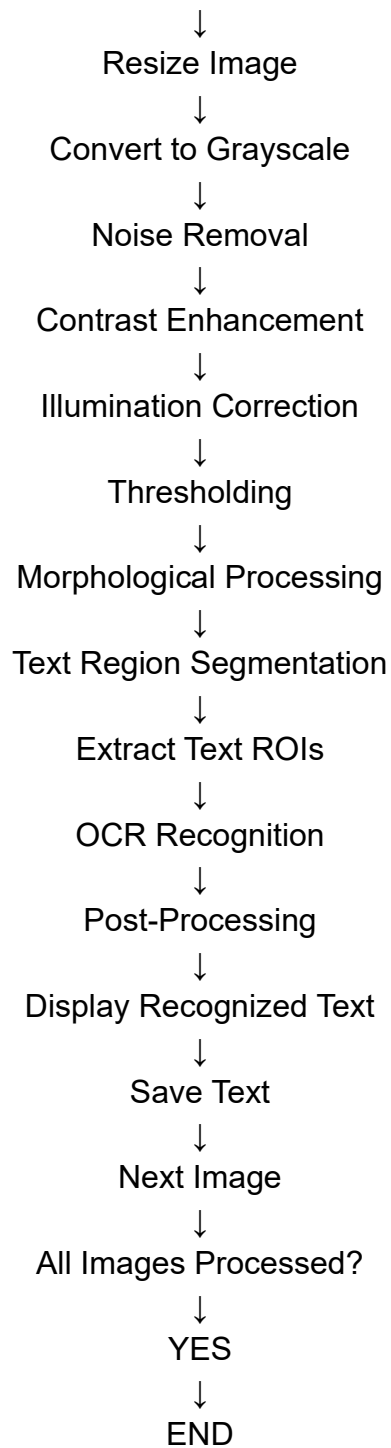
START



Load Input Image Folder



Read Image



OpenCV Functions Used

Stage	OpenCV Function / Concept
Read image	cv2.imread()
Resize	cv2.resize()
Grayscale conversion	cv2.cvtColor()
Noise removal	cv2.GaussianBlur() / cv2.medianBlur()
Contrast enhancement	cv2.equalizeHist()

Stage	OpenCV Function / Concept
Local enhancement	cv2.createCLAHE()
Thresholding	cv2.threshold()
Adaptive thresholding	cv2.adaptiveThreshold()
Morphological processing	cv2.morphologyEx()
Text segmentation	cv2.findContours()
Region extraction	Array slicing / ROI
OCR	Tesseract or another OCR engine

Handling the Constraints

1. Noisy Images

Gaussian or median filtering removes small noise and unwanted pixels before OCR.

2. Low Contrast

Histogram equalization or CLAHE improves the contrast between text and background.

3. Uneven Illumination

Adaptive thresholding applies different thresholds to different image regions, making it suitable for documents with shadows or uneven brightness.

4. Batch Processing

The FOR loop processes every image in the input folder sequentially. This avoids loading all images into memory at the same time.

5. Efficient Processing

Only the detected text regions are passed to the OCR engine. This reduces unnecessary computation and improves processing speed.

Conclusion

The proposed OCR pipeline consists of image acquisition, resizing, grayscale conversion, noise removal, contrast enhancement, thresholding, morphological processing, text segmentation, OCR recognition, and post-processing. The system handles noisy and low-contrast documents using filtering and enhancement techniques. Adaptive thresholding improves performance under uneven illumination, while batch processing allows multiple documents to be processed efficiently with controlled memory usage.

3. Motion Detection and Tracking System A

surveillance system must:

- A. Read video input
 - B. Detect motion between consecutive frames
 - C. Track moving objects
 - D. Highlight detected motion regions
- Constraints:

- Should handle dynamic background
- Must operate in near real-time

Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.

ANS:

1. Motion Detection and Tracking System

Objective

Design a motion detection and tracking system that continuously reads a video stream, detects motion between consecutive frames, tracks moving objects, and highlights motion regions. The system should handle dynamic backgrounds and operate in near real-time.

Pseudocode

ALGORITHM MotionDetectionAndTracking

INPUT:

Live video stream or video file

OUTPUT:

Video with highlighted moving objects and tracking information

BEGIN

MODULE 1: INITIALIZATION

Import OpenCV library

Open video source

video ← cv2.VideoCapture(input_source)

Create background subtractor

```
background_model ←
    cv2.createBackgroundSubtractorMOG2(
        history = 500,
        varThreshold = 50,
        detectShadows = TRUE
    )
```

Initialize object tracker

tracker ← InitializeTracker()

Set minimum contour area

MIN_AREA $\leftarrow$ 500

IF video cannot be opened THEN

Display "Video input error"

STOP

END IF

MODULE 2: CONTINUOUS VIDEO PROCESSING

WHILE video is running DO

Read current frame

success, frame $\leftarrow$ video.read()

IF success = FALSE THEN

BREAK

END IF

MODULE 3: FRAME PREPROCESSING

Resize frame if required

frame $\leftarrow$ cv2.resize(frame, desired_size)

Convert frame to grayscale

gray $\leftarrow$ cv2.cvtColor(
frame,
BGR_TO_GRAY
)

Apply Gaussian filtering

filtered $\leftarrow$ cv2.GaussianBlur(
gray,
(5,5),
0
)

MODULE 4: FRAME DIFFERENCING

Compare current frame with previous frame

IF previous_frame does not exist THEN

 previous_frame ← filtered

 CONTINUE

END IF

frame_difference ← cv2.absdiff(
 previous_frame,
 filtered
)

Apply threshold

motion_mask ← cv2.threshold(
 frame_difference,
 25,
 255,
 BINARY
)

previous_frame ← filtered

MODULE 5: DYNAMIC BACKGROUND HANDLING

Apply adaptive background subtraction

foreground_mask ←
 background_model.apply(frame)

Combine or select the appropriate
motion mask and foreground mask

Remove shadows if required

MODULE 6: NOISE FILTERING

Apply morphological opening

motion_mask ←
 MorphologicalOpening(motion_mask)

Apply morphological closing

```
motion_mask ←  
    MorphologicalClosing(motion_mask)
```

Apply dilation to connect

nearby motion regions

```
motion_mask ←  
    cv2.dilate(  
        motion_mask,  
        kernel,  
        iterations = 2  
    )
```

MODULE 7: MOTION REGION DETECTION

Find contours in motion mask

```
contours ← cv2.findContours(  
    motion_mask  
)
```

FOR each contour in contours DO

Calculate contour area

IF contour area < MIN_AREA THEN

Ignore contour

ELSE

Calculate bounding box

```
x, y, width, height ←  
    cv2.boundingRect(contour)
```

Store motion region

END IF

END FOR

MODULE 8: OBJECT TRACKING

IF new motion regions are detected THEN

FOR each detected region DO

Initialize or update tracker
using bounding box

END FOR

ELSE

Update existing trackers

END IF

FOR each tracked object DO

Update object position

Assign object ID

Store current position

END FOR

MODULE 9: VISUALIZATION

FOR each tracked object DO

Draw bounding rectangle

Draw object ID

Draw tracking path if required

END FOR

Display motion mask

Display processed video frame

Display "Motion Detected"
when moving objects are present

MODULE 10: REAL-TIME CONTROL

Show processed frame

Wait for a very short time

IF user presses 'q' THEN
 BREAK
END IF

END WHILE

MODULE 11: TERMINATION

Release video source

Destroy all OpenCV windows

Display "Motion Detection Stopped"

END

Frame Differencing Logic

Previous Frame



Current Frame



Absolute Difference



Thresholding



Motion Mask



Noise Filtering

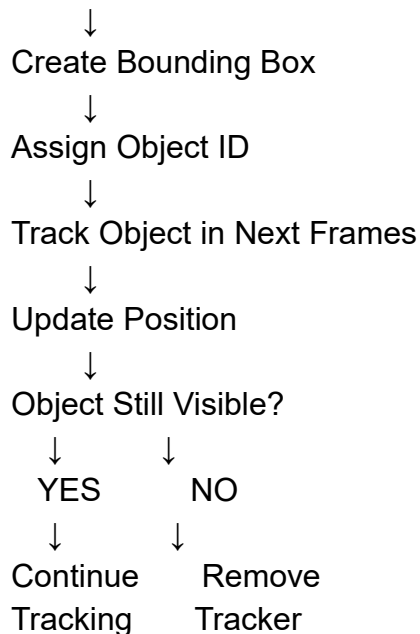


Motion Regions

The absolute difference between consecutive frames identifies pixels that have changed. A threshold converts these changes into a binary motion mask.

Object Tracking Logic

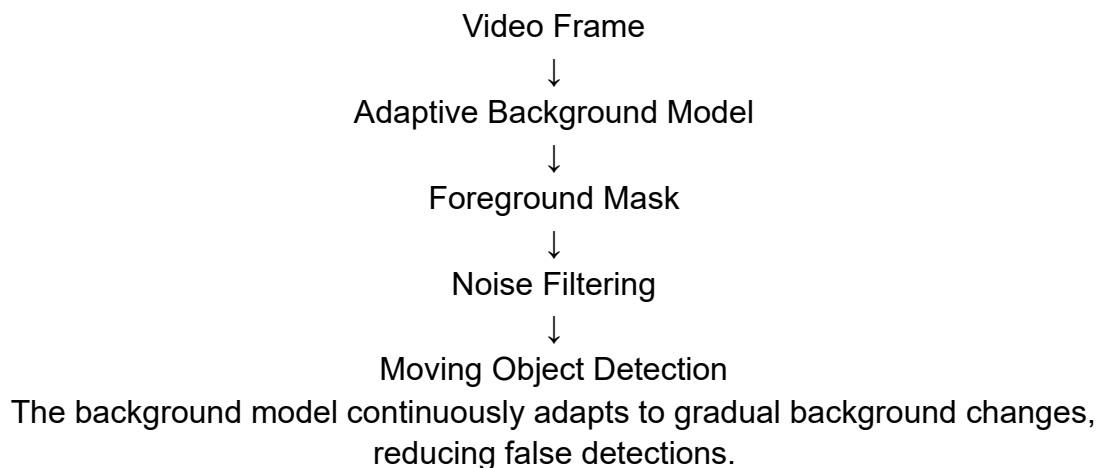
Motion Region Detected



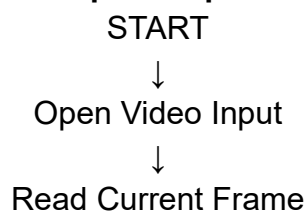
Tracking prevents the system from treating the same moving object as a completely new object in every frame.

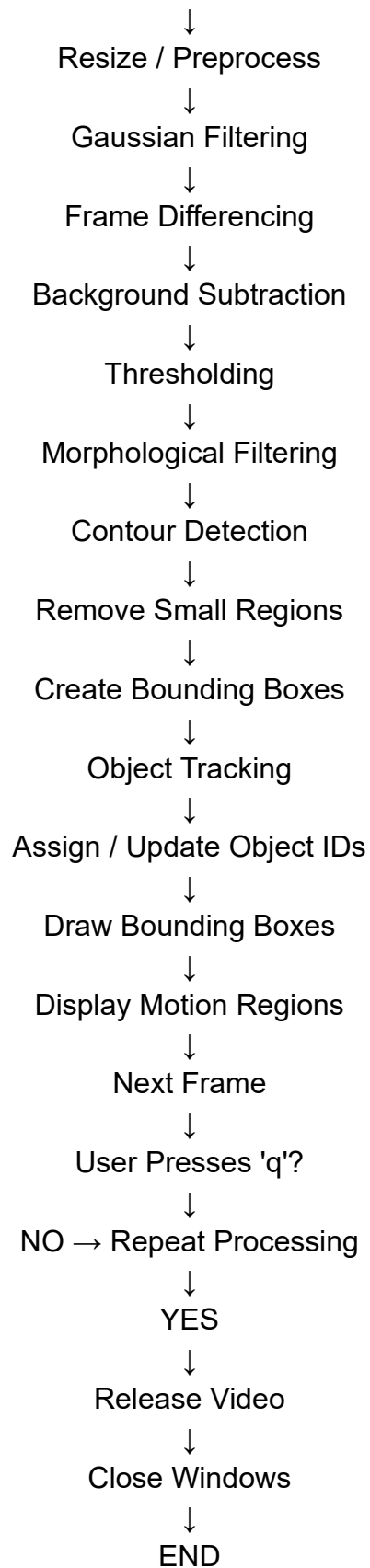
Dynamic Background Handling

Dynamic backgrounds such as moving trees, rain, shadows, and changing illumination can produce false motion. To handle this, an adaptive background subtraction method such as MOG2 is used.



Complete Pipeline





Handling the Constraints

Requirement	Solution
Read video input	cv2.VideoCapture()
Detect motion	Frame differencing using cv2.absdiff()
Dynamic background	MOG2 adaptive background subtraction
Remove noise	Gaussian blur and morphological operations
Remove false detections	Minimum contour-area filtering
Track moving objects	Object tracker with bounding boxes and IDs
Highlight motion	cv2.rectangle() and tracking paths
Near real-time	Lightweight OpenCV operations and frame resizing
Continuous video	WHILE loop processes frames continuously

Conclusion

The proposed motion detection and tracking system continuously reads video frames and detects changes using frame differencing. Gaussian filtering and morphological operations reduce noise, while MOG2 background subtraction handles dynamic backgrounds. Contours are used to identify significant motion regions, and object tracking maintains the position and identity of moving objects across frames. Bounding boxes, object IDs, and motion masks provide clear visualization. The lightweight processing pipeline allows the system to operate in near real time.

Code of Conduct:

I G.Joel Richard (192524054) certify that this submission is my original work and that I have *adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear

Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient
--------------	-----	---------------------------------------	---	--------------------------	---------------------------